FULL-STACK DEFI GOVERNANCE

# Astra Protocol

A governance-oriented DeFi application combining smart contracts, on-chain DAO mechanics, a working frontend, and indexed blockchain data.

Solidity | Foundry | Next.js | The Graph | Arbitrum Sepolia

PROJECT GOAL

# Decentralized governance from the ground up

Token holders participate in governance and directly influence protocol behavior through on-chain voting.

CONTRACTS

- `GovernanceToken`
- `ProtocolGovernor`
- `ProtocolTimelock`
- `Treasury`

INFRASTRUCTURE

- `Frontend dApp`
- `Subgraph (The Graph)`
- `Testnet Deployment`
- `Full Test Suite`

ARCHITECTURE

# Repository structure

Modular layout with clear separation of responsibilities.

SRC/

- governance/
- token/
- amm/
- vault/
- oracle/

OTHER MODULES

- test/ (5 categories)
- script/ (deploy)
- astra-frontend/
- subgraph/

GOVERNANCE

# Governance architecture

Three-layer governance based on the standard DAO pattern used in production systems.

→ **GovernanceToken** — ERC20Votes model; voting power activated through delegation

→ **ProtocolGovernor** — proposal creation, voting, quorum checks, proposal lifecycle

→ **ProtocolTimelock** — mandatory delay between a successful vote and execution

DEFI MODULES

# Treasury, AMM, and Vault

The protocol is more than a governance demo — it integrates real DeFi primitives.

`Treasury` `AMMFactory` `AMMPair` `LPToken`

`YieldVault (ERC-4626)` `ChainlinkOracle`

The vault uses the **ERC-4626** tokenized vault standard. The AMM follows the **constant-product** model. Chainlink provides tamper-resistant external pricing.

FRONTEND

# Frontend overview

Built with Next.js, Wagmi, RainbowKit, and ethers. A fully functional dApp, not just a visualization layer.

`Wallet connection` `Contract reads` `State-changing txs`

`Proposal UI` `Network detection` `Error feedback`

FRONTEND

# Wallet connection and read functions

Supports MetaMask and WalletConnect. Once connected, the user can read:

- `Governance token balance`
- `Voting power`
- `Delegate address`
- `Vault total assets`
- `Personal vault shares`

FRONTEND

# Write functions

Users can participate in governance directly from the UI — no separate tooling needed.

→ **Delegate** — assign voting power to a delegate address

→ **Create Proposal** — submit a new on-chain governance proposal

→ **Cast Vote** — vote on an active proposal; state is checked before enabling the button

FRONTEND

# Proposal list, network check, and error handling

Each proposal shows its current state. Wrong network → user is prompted to switch to Arbitrum Sepolia. All errors are shown in plain language.

```
Pending  →  Active  →  Succeeded  →  Queued  →  Executed
```

`Defeated` `Insufficient power` `Wrong network` `Gas error`

TESTING

# Testing strategy

The full test suite uses Foundry and covers five categories to validate contracts at multiple levels.

TEST TYPES

- Unit tests
- Integration tests
- Invariant tests
- Security tests
- Fork tests

COVERAGE

**97.16%**
line coverage

**100%**
function coverage

TESTING

# Governor tests

`Governor.t.sol` verifies the full governance lifecycle.

- → Voting power is active after delegation
- → Proposals can be created with enough power; rejected without it
- → Proposal state transitions are tracked correctly
- → Voting is only allowed after the voting delay
- → Double voting is blocked

SECURITY

# Coverage and security audit

| 0 | 0 | 0 |
| --- | --- | --- |
| Critical findings | High findings | Medium findings |

Access control

Checks-effects-interactions

Oracle staleness

Governance safety

DEPLOYMENT

# Deployed to Arbitrum Sepolia

Deployment scripts and contract records are included in the repository. The project runs on a real testnet, making frontend interaction and verification realistic rather than purely local.

```
Compile  →  Deploy script  →  Arbitrum Sepolia  →  Frontend
```

THE GRAPH

# Subgraph and indexed data

A subgraph was deployed to The Graph Studio to index governance and vault events via GraphQL.

| INDEXED EVENTS | SCHEMA ENTITIES |
| --- | --- |
| `Proposal creation` | `Proposal` |
| `Voting` | `Vote` |
| `Delegation` | `Delegation` |
| `Vault deposits` | `TokenHolder` |
| `Vault withdrawals` | `VaultDeposit / VaultWithdrawal` |

THE GRAPH

# Frontend + The Graph integration

The dApp uses a hybrid data strategy — combining live contract reads with subgraph queries.

- → **Live reads** — wallet balance, voting power, vault shares fetched directly from the chain
- → **Subgraph reads** — historical proposal and delegation data loaded via GraphQL
- → Reflects the architecture used in real Web3 applications

CHALLENGES

# Main engineering challenges

→ Aligning frontend actions with on-chain governance rules and contract states

→ Deploying the subgraph with compatible ABI files and schema definitions

→ Synchronizing frontend GraphQL queries with the actual subgraph structure

In practice, integration work is often as complex as writing the contract code itself.

OUTCOME

# Project outcome

A complete governance-focused blockchain application covering the full development pipeline.

Smart contracts | DAO governance | Tested workflow

Arbitrum Sepolia | Frontend dApp | Subgraph indexing

CONCLUSION

# Thank you

Astra Protocol demonstrates how a decentralized application can be designed not only at the smart contract level, but as a complete product — with testing, deployment, frontend usability, and indexed data support.

`Governance logic` `Smart contracts` `Frontend` `Subgraph`